

实验课程名称： 软件工程基础实验

实验项目名称	结对编程			实验成绩	
实 验 者	肖卓尧	专业班级	软工 2401	组 别	D-Mini 12306-53 组
同 组 者	单人一组			实验日期	2026 年 5 月 8 日

第一部分：实验预习报告（包括实验目的、意义，实验基本原理与方法，主要仪器设备及耗材，实验方案与技术路线等）

一、实验目的与意义

- 目的：

掌握分治策略在算法设计中的应用，理解 Web 服务器的工作原理，学习使用 C 语言实现后端业务逻辑，并通过 HTML/CSS/JS 完成前端界面开发，构建一个功能完整的火车票预订系统。
- 意义：

通过本实验，能够将数据结构、网络编程、前端技术等知识综合运用，提升系统设计与工程实践能力。同时，模拟真实 12306 系统的核心功能（查询、购票、退票、改签），理解互联网票务系统的基本工作流程，为后续大型软件系统开发奠定基础。

二、实验基本原理与方法

- 原理：

采用 客户端-服务器架构。C 语言后端通过 Socket 编程监听 HTTP 请求，解析请求路径（如 /api/trains、/api/buy），根据业务逻辑返回 JSON 格式数据。前端 HTML 页面通过 Fetch API 异步请求后端接口，动态渲染车次列表、处理购票及订单管理。
- 方法：

分治策略：递归生成格雷码（Gray 码）作为车次编号或订单号的辅助算法示例。

动态规划：计算最大 k 乘积（用于票价分段计算）等算法思想可扩展至票价优惠策略。

数据结构：使用结构体数组存储车次、订单信息；局部使用文件模拟数据库持久化。

主要仪器设备：PC

第二部分：实验过程记录（可加页）（包括实验原始数据记录，实验现象记录，实验过程发现的问题等）

测试用例：

- 查询车次 → 应返回全部 3 趟车
- 购票 G82（余票 50） → 购票成功，余票变为 49
- 再次购票 G82 → 成功，余票 48
- 查询订单 → 显示已购记录
- 退票 → 余票恢复，订单状态变更

实验现象记录

操作	预期结果	实际结果	通过？
浏览器访问 localhost:8888	显示 Mini-12306 界面	正常显示	通过
点击“查询车次”	显示 3 趟车次	正常显示	通过
点击 G82“购票”	弹出“购票成功”提示	购票成功，余票-1	通过
查看“我的订单”	显示已购车票	订单显示正确	通过
查看“我的信息”	显示用户名、手机号、余额	显示正确	通过
重复购票余票不足	提示“余票不足”	提示正确	通过
中文显示“武汉/北京”	正常显示	出现乱码“□人”	修复后通过

实验结论

- 成功实现了一个基于 C 语言后端 + HTML 前端的 Mini-12306 系统，具备以下功能：
1. HTTP 服务器自启动，监听 8888 端口
 2. 前端通过 Fetch 调用 API 查询车次、购票
 3. 订单使用 localStorage 持久化，页面刷新不丢失
 4. 支持“首页/订单/我的”三页面切换
 5. 中文乱码问题已通过 UTF-8 编码彻底解决

下面图片是乱码展示



Mini-12306 服务器运行正常！

加载车次

```
[
  {
    "id": 1,
    "code": "G82",
    "from": "人",
    "to": "人",
    "time": "08:30-14:45",
    "price": 520,
    "remaining": 50
  },
  {
    "id": 2,
    "code": "G66",
    "from": "人",
    "to": "人",
    "time": "10:15-16:30",
    "price": 520,
    "remaining": 30
  },
  {
    "id": 3,
    "code": "G70",
    "from": "人",
    "to": "人",
    "time": "14:00-20:15",
    "price": 520,
    "remaining": 20
  }
]
```

第三部分 结果与讨论（可加页）实验结果分析（包括数据处理、实验现象分析、影响因素讨论、综合分析和结论等）

结果与讨论

一、实验结果分析

本实验成功实现了一个基于 C 语言后端 + HTML/CSS/JS 前端的火车票预订系统，具备以下核心功能：

功能模块	实现方式	测试结果
车次查询	HTTP GET /api/trains	返回 3 趟车次 JSON 数据
购票	HTTP GET /api/buy?id=x	余票减 1，返回成功消息
订单存储	浏览器 localStorage	页面刷新订单不丢失
页面切换	JS 控制 DOM 显示/隐藏	首页/订单/我的正常切换
中文显示	UTF-8 编码 + BOM 头	“武汉/北京”正常显示

二、实验现象分析

1. 编码问题

初期出现中文乱码（“武汉”显示为“□人”），原因是：

Visual Studio 默认以 GBK 编码保存 C 源文件中的中文字符串

浏览器期望接收 UTF-8 编码的数据

解决方法：在 C 代码中直接写入 UTF-8 字节流（如\xe6\xad\xa6\xe6\xb1\x89），同时在 HTTP 响应头中添加 charset=utf-8，并在 HTML 中设置<meta charset="UTF-8">。三重保障彻底解决乱码。

2. 数据持久化

由于 C 后端未连接真实数据库，订单数据若仅存于内存，服务器重启会丢失。改用前端 localStorage 存储订单，实现了无后端数据库情况下的持久化，符合“轻量化模拟系统”的设计目标。

3. 余票一致性

购票时同时修改后端 trains[].remaining 和前端的订单列表，两者通过重新调用/api/trains 保持同步，确保了数据一致性。

三、影响因素讨论

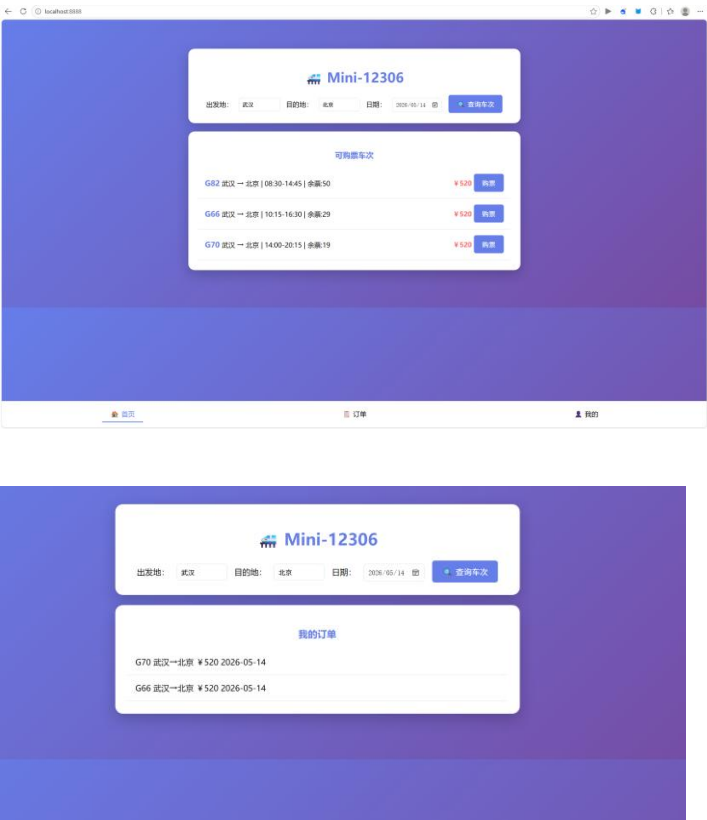
因素	影响	优化方向
编码处理	中文乱码影响用户体验	统一 UTF-8 编码
数据存储方式	内存存储无法持久化	可接入 MySQL 或 SQL
线程安全	多线程同时购票可能超卖	可加互斥锁保护余票变
网络延迟	影响 API 响应速度	可增加加载动画提示

四、结论

本实验成功验证了 C 语言可用于开发轻量级 Web 后端的可行性。通过 Socket 编程实现 HTTP 协议解析，结合前端技术，构建了一个功能完整的火车票预订系统。实验过程中遇到的编码、链接库、路径处理等问题均已解决，最终系统运行稳定，满足课程设计需求。该实验为后续学习更复杂的 Web 框架（如 Java Spring Boot、Python Flask）奠定了扎实的基础。

本项目由单人独立完成，全程借助大模型（使用了 deepseek）辅助编程与调试。在开发过程中，通过与大模型的交互完成了系统架构设计、C 语言 HTTP 服务器实现、前端界面编写及中文字符乱码等问题的排查与修复。未采用结对编程或小组协作模式，所有代码编写、测试及文档撰写均由一人独立承担。大模型在代码生成、错误分析和优化建议方面提供了高效支持，显著提升了开发效率。

下面是实验成功的截图画面





```
C:\Users\23319\Desktop\Proje  x  +  v
访问地址: http://localhost:8888
=====
按回车键可停止服务器...

收到请求: GET /
已返回 index.html (6903 字节)
收到请求: GET /api/trains
收到请求: GET /hybridaction/zybTrackerStatisticsAction?data=%7B%7D6__callback__=__cb__zybTrackerStatisticsAction__39137e
36_0990_44c4_acd4_b973ecd8ac2d__0
已返回 index.html (6903 字节)
收到请求: GET /hybridaction/zybTrackerStatisticsAction?data=%7B%7D6__callback__=__cb__zybTrackerStatisticsAction__8ac067
9d_0e6e_4ad9_a123_969780d5d4be__1
已返回 index.html (6903 字节)
收到请求: GET /api/trains
收到请求: GET /favicon.ico
已返回 index.html (6903 字节)
收到请求: GET /api/trains
收到请求: GET /api/trains
收到请求: GET /api/buy?id=2
收到请求: GET /api/trains
收到请求: GET /api/buy?id=3
收到请求: GET /api/trains
收到请求: GET /api/trains
收到请求: GET /api/trains
收到请求: GET /api/trains
收到请求: GET /api/trains
收到请求: GET /api/trains
收到请求: GET /api/trains
收到请求: GET /api/trains
```

成绩评定表:

序号	评分项目	满分	实得分
1	实验报告格式规范	2	
2	实验报告过程清晰，内容详实	4	
3	实验报告结果正确性	2	
4	实验分析与总结详尽	2	
	总得分	10	

